

Logic Explanation

Approach, design decisions and validation for E3, M4 and H4

Round 1, Phase 2: Analytical Core

Competition: Data Vortex | AARUUSH'26

Theme: Rebuilding the Social Engine

Team: Entropy

Members: Saanvi Grover & Aditya Sharma

Questions: E3 · M4 · H4

Database: SQLite (Entropy_social_engine.db)

Why These Three Questions

We chose one question per level that (a) needs a different SQL technique, (b) forces an explicit decision about the missing values left by the corruption, and (c) can be tested against a benchmark inside the query instead of only returning rows.

Question	Core technique	Benchmark built into the query
E3 Average engagement by platform	GROUP BY, RANK(), cross-joined grand mean	Each platform's deviation from the overall mean
M4 Platform behaviour, 30k+ followers	Join, conditional aggregation in one scan, two independent RANK()s	Same platform's average for users under 30k followers
H4 Follower-to-engagement anomaly	Four-level CTE chain, per-user imputation, two NTILE(10) rankings, PERCENT_RANK()	Per-post decile, separating volume from performance

Shared Design Decisions

Missing values: one rule per question, each justified

The cleaned data keeps NULLs where the corruption destroyed a value: `platform` (1,784 posts) and `likes` (1,814 posts, 260 of them also missing platform). Phase 1 showed this corruption is Missing Completely at Random (Mann–Whitney U, $p = 0.64$), so dropping or imputing does not bias results in expectation. The right rule still depends on what is being computed:

Query	Rule	Why
E3	Exclude only posts with a missing platform; let <code>AVG()</code> skip missing likes.	Follows the question literally. Shares and comments are averaged over every post; total engagement ignores posts without likes, which is the brief's own definition (E2).
M4	Exclude posts with a missing platform or missing likes.	The question is about engagement per post, so the brief's definition of engagement applies to both groups.
H4	Replace a missing like count with that user's own average likes (global average for the 2 users with none).	A user total is a sum. Zero-filling would remove about 2,500 engagement per corrupted post, penalising whichever users the corruption hit most.

Deterministic, benchmarked output

Rankings use `RANK()` (ties stay visible) and the H4 deciles add `user_id` as a tie-break, so the same data always gives the same list. Every query returns a comparison column (`pct_vs_overall`, `lift_pct`, `per_post_decile`), because “which is highest” is only meaningful next to “by how much, compared with what”.

Index-aware design

`EXPLAIN QUERY PLAN` confirms the schema's indexes are used: `idx_posts_platform` drives the platform grouping in E3 and M4, `idx_posts_user_id` drives the per-user roll-ups and joins in H4, and each join to `users` is a primary-key lookup.

E3 – Average Engagement by Platform

Approach

1. **platform_stats**: filter out posts with no platform, GROUP BY platform, and compute post count, count of posts with likes, and the four averages.
2. **overall**: the grand mean of total engagement over the same posts, a single value attached to every row with CROSS JOIN.
3. **Final SELECT**: RANK() by average total engagement plus pct_vs_overall.

Why the literal reading

The question asks for average likes, shares and comments per platform and says nothing about dropping posts with missing likes. Filtering them out would silently change the shares and comments averages (about 1,550 posts removed). SQL's own NULL behaviour gives the literal answer for free: `AVG(likes)` skips missing values, and `AVG(likes + shares + comments)` skips posts whose total is undefined.

A consequence worth knowing: average total engagement (Instagram 4,040.0) is not exactly the sum of the three component averages (4,041.5), because the likes and total averages cover 1,693 posts while shares and comments cover all 1,989.

Why add a benchmark and a power check

Ranking alone hides the size of the gaps. `pct_vs_overall` shows Instagram leads by +0.78%. To tell “no difference” apart from “too little data to see one”, we compute the minimum detectable difference between two platforms at 80% power and $\alpha = 0.05$:

$$\text{MDE} = 2.8 \sigma \sqrt{2/n} = 2.8 \times 1,576 \times \sqrt{2/1,732} \approx 150 \text{ (3.7\% of the mean).}$$

Validation

- Posts sum to 10,216 = 12,000 – 1,784 missing platforms; posts with likes sum to 8,662 = 10,216 – 1,554.
- Kruskal–Wallis across platforms: $H = 2.97$, $p = 0.56$. The largest gap (88) is below the detectable 150.

M4 – Platform Behaviour by High-Follower Users

Approach

1. **post_engagement**: join posts to users, flag each post `is_high_follower` ($\geq 30,000$ followers) and compute its engagement.
2. **by_platform**: one GROUP BY platform with conditional aggregation, `AVG(CASE WHEN is_high_follower THEN engagement END)` and the same for the rest, so both groups come from a single scan. `SUM(is_high_follower)` counts cohort posts because SQLite stores booleans as 0/1.
3. **Final SELECT**: rank platforms separately for each group and report `lift_pct`, the cohort's percentage difference from users under 30k.

Why compare with users under 30k, not all users

The 594 high-follower users are 39.6% of all users. An “all users” baseline would contain the cohort itself and pull toward it, shrinking any real gap. Comparing two disjoint groups measures the effect of audience size directly.

Why rank both groups

If large accounts really had a best platform, the platform order should be broadly the same in both groups. Ranking each separately tests that in the output itself.

Validation

Instagram's lift has a raw p of 0.023, but five platforms were tested, so we apply Holm's correction for multiple comparisons:

Platform	Lift	Raw p (Mann–Whitney)	Holm-adjusted p
Instagram	+4.42%	0.023	0.116
YouTube	−3.50%	0.061	0.245
Twitter	+1.10%	0.455	1.000
Reddit	−0.92%	0.536	1.000
Facebook	−0.61%	0.676	1.000

- 30k+ versus under 30k across all platforms: Mann–Whitney $p = 0.98$.
- Platform differences within the cohort: Kruskal–Wallis $p = 0.18$.
- Agreement between the two groups' rankings: Spearman $\rho = -0.30$ (YouTube is 1st for smaller accounts and 5th for large ones).

H4 – Follower-to-Engagement Anomaly

Approach: four levels of analysis

1. **Impute** (`user_like_avg`, `global_like_avg`, `post_engagement`): `COALESCE(p.likes, ula.avg_likes, g.avg_likes)` fills a missing like count with the author's own average, falling back to the global average. `likes_imputed` records which posts were filled.
2. **Aggregate** (`user_engagement`): roll posts up to users for post count, imputed posts, total engagement and average per post.
3. **Rank** (`ranked_users`): `NTILE(10)` by total gives the top 10% (exactly 150 of 1,500 users); a second `NTILE(10)` by per-post engagement shows whether a user's posts are strong or just numerous; `PERCENT_RANK()` places each user precisely.
4. **Filter and classify**: keep the top total decile with fewer than 5,000 followers, and label each survivor *Exceptional per post* (per-post decile 1) or *Volume-driven*.

Why impute instead of zero-filling

We ran both versions. Zero-filling changed the answer: `user_hvqyzo`i and `user_06v0exkg` would have been listed, while `user_zozhwbis` and `user_zwtxksrs` would have been missed. `user_zwtxksrs` has 4 of 13 posts with corrupted likes and falls below the threshold only because those posts count as zero likes. Two of 16 names (12.5%) depending on how corruption is handled is too much for a list meant to flag accounts.

Why NTILE rather than a fixed threshold

A condition like `total_engagement > 48159` is correct only for today's data. `NTILE(10)` defines the top 10% relative to the population, so the query stays correct when data changes.

Why a second ranking by per-post engagement

Total engagement is posts $\times$ average per post, so a top-decile total can come from either. The per-post decile separates the two, turning a list of 16 names into a distinction the brief asks for: “unusually high performing or suspicious”.

Validation

- Top decile: 150 users; threshold 48,159.
- 131 users (8.73%) have fewer than 5,000 followers. If followers are irrelevant, $150 \times 0.0873 = 13.1$ should reach the top decile; 16 do (binomial $p = 0.38$). Followers versus total engagement: Spearman $\rho = -0.01$ ($p = 0.64$).
- Posts versus total engagement: Spearman $\rho = 0.91$. The 16 users average 12.9 posts against 8.0 for all users.
- 2 of 16 (12.5%) are also top 10% per post, compared with 15.3% of the whole top decile.

Tooling

SQLite 3 via Python `sqlite3` and `pandas` in `Entropy_01_SQL_Analysis.ipynb`; statistical checks with `scipy.stats`; output screenshots captured from Jupyter running the query files; charts drawn with `matplotlib` from the same query results.